Workshop no.2 2026-2027	Module	MTI Methods and techno. of Impl.
	Branch	Master GSI 1st year
Chapter 2: OOP	ملاحظة :	

Part 1: Python + OOP الجزء الأول: بايثون والبرمجة كائنية التوجه

Objectives : Handle OOP using Python البرمجة الكائنية

Data : M1 GSI canvas منهاج السنة الأولى ماستر تخصص هندسة أنظمة الإعلام الآلي

Tools : Python compiler

You have the Master 1 GSI canvas (see appendix), please analyze it to extract all possible classes.
تجد في الملحق منهاج الماستر 1 تخصص هندسة أنظمة الإعلام الآلي ،(GSI) حلّله لاستخلاص الفئات الممكنة (الأصناف).

1. What are the entities in our academic model? ما هي الكيانات (Entities) في المنهاج؟
2. First we work on the `Module` class, then test this code. ابدأ بالعمل على فئة المادة `Module` وجرّب الكود.

```python
class Module:
    """
    Represents a teaching module with pedagogical and evaluation attributes.
    """

    def __init__(
        self,
        name: str="",  # module code
        title: str="",
        coef: int=1,
        credit: int=1,
        hours_lecture: float=1.5,
        hours_td: float=0,
        hours_tp: float=0,
        teaching_mode: str="In-person",    # "Distance" or "In-person"
        continous_percent:int=40,
        exam_percent: int=60
    ):
        self._WEEKS = 15  #private in python preceded by _
        # Basic attributes
        self.name = name
        self.title = title
        self.coef = coef
        self.credit = credit

        # Workload distribution (volume horaire)
        self.hours_lecture = hours_lecture
        self.hours_td = hours_td
        self.hours_tp = hours_tp

        # Teaching and evaluation mode
        self.teaching_mode = teaching_mode # "Distance" or "In-person"
        self.evaluation_continous_percent = continous_percent
        self.evaluation_exam_percent = exam_percent

        # Total hours x semesters
```

```
        self.total_hours = self._WEEKS * (self.hours_lecture +
                    self.hours_td+ self.hours_tp)

        self._grades = {"tp": None, "td": None, "exam": None}

    # Encapsulation: controlled access to grades
    def set_grade(self, tp=None, td=None, exam=None):
        if tp is not None:
            self._grades["tp"] = tp
        if td is not None:
            self._grades["td"] = td
        if exam is not None:
            self._grades["exam"] = exam

    def summary(self):
        """Return a short textual description of the module."""
        return (
            f"Module: {self.title} ({self.name})\n"
            f"Coefficient: {self.coef}, Credits: {self.credit}\n"
            f"Hours: total {self.total_hours}, {self.hours_lecture} Lecture,
             {self.hours_td} TD, "
            f"{self.hours_tp} TP\n"
            f"Teaching mode: {self.teaching_mode}, "
            f"Evaluation mode: Continous: {self.evaluation_continous_percent}%
             Exam: {self.evaluation_exam_percent}% "
        )

    def calculate_average(self):
        """to be implemented later."""
        pass
# Example usage
if __name__ == "__main__":
    mti = Module(
        name="MTI",
        title="Methods and Technologies of Implementation",
        coef=3,
        credit=5,
        hours_lecture=1.5,
        hours_tp=1.5,
        teaching_mode="In-person"
    )

    print(mti.summary())
```

3. Implement `calculate_average()` and `calculate_credits()`. أنجز دالتي حساب المعدل والرصيد.

```
def calculate_average(self):
    """Polymorphism: different modules could override this behavior."""
    tp = self._grades["tp"]
    td = self._grades["td"]
    exam = self._grades["exam"]
    percent_exam = self.evaluation_exam_percent
    percent_tp = percent_td = 0
    if self.hours_tp and self.hours_td: # not null
        percent_tp = percent_td = self.evaluation_continous_percent/2
    elif self.hours_td:
```

```
            percent_td = self.evaluation_continous_percent
        elif self.hours_tp:
            percent_tp = self.evaluation_continous_percent
        return (
            tp * percent_tp / 100 +
            td * percent_td / 100 +
            exam * percent_exam / 100
        )
    def calculate_credits(self):
        """Polymorphism: different modules could override this behavior."""
        avg = self.calculate_average()
        if avg >= 10 :
            return self.credit
        else:
            return 0
```

4. Implement the `Unit` class. أنجز فئة الوحدة `Unit`.

5. Identify similarities between `Unit` and `Module`. حدد أوجه التشابه بين فئتي المادة والوحدة `Unit` و `Module`.

6. Explain inheritance relationship between `Unit` and `Module`. اشرح علاقة الوراثة بين الفئتين `Unit` و `Module`.

7. We opt for abstraction of the two classes into an abstract class named `AcademicElement`. Test the following code. نختار تجريد الفئتين في صنفٍ تجريديٍّ واحد يسمى `AcademicElement`. اختبر الكود التالي.

`AcademicElement`

```

from abc import abstractmethod
class AcademicElement:
    """Abstract base class for academic entities (Module, Unit, Semester, etc.)."""

    def __init__(self, name, title):
        self.name = name
        self.title = title
        self._WEEKS = 15  # Encapsulation: private attribute
        self.coef = 0
        self.credit = 0

        # Workload distribution (volume horaire)
        self.hours_lecture = 0
        self.hours_td = 0
        self.hours_tp = 0

    @abstractmethod
    def calculate_average(self):
        """Abstract method to be implemented by subclasses."""
        pass
    @abstractmethod
    def calculate_credits(self):
        """Abstract method to be implemented by subclasses."""
        pass
```

`Module`

```
class Module(AcademicElement):
    """
```

```
    Represents a teaching module with pedagogical and evaluation attributes.
    """

    def __init__(
        self,
        name: str="",  # module code
        title: str="",
        coef: int=1,
        credit: int=1,
        hours_lecture: float=1.5,
        hours_td: float=0,
        hours_tp: float=0,
        teaching_mode: str="In-person",    # "Distance" or "In-person"
        continous_percent:int=40,
        exam_percent: int=60
    ):
        super().__init__(name, title)
        self.coef = coef
        self.credit = credit

        # Workload distribution (volume horaire)
        self.hours_lecture = hours_lecture
        self.hours_td = hours_td
        self.hours_tp = hours_tp

        # Teaching and evaluation mode
        self.teaching_mode = teaching_mode # "Distance" or "In-person"
        self.evaluation_continous_percent = continous_percent
        self.evaluation_exam_percent = exam_percent

        # Total hours x semesters
        self.total_hours = self._WEEKS * (self.hours_lecture +
                self.hours_td+ self.hours_tp)

        self._grades = {"tp": 0, "td": 0, "exam": 0}

    # Encapsulation: controlled access to grades
    def set_grade(self, tp=None, td=None, exam=None):
        ... # same as previous

    def summary(self):
        """Return a short textual description of the module."""
        ... # same as previous
```

Usage:

```
# Example usage (for demonstration)
if __name__ == "__main__":
    m1 = Module("MTI", "Methods & Technologies of Implementation", coef=3, credit=5,
            hours_tp=1.5, continous_percent=40, exam_percent=60)
    m1.set_grade(tp=10, exam=12)
    print("Module average:", m1.name, m1.calculate_average())
```

Unit

```python

class Unit(AcademicElement):
    """Represents a teaching unit containing multiple modules."""

    def __init__(self, name, title, modules=[]):
        super().__init__(name, title)
        self._modules = modules     # private here is not necessary, used as example for

    def add_module(self, module):
        self._modules.append(module)
```

Usage:

```python
# Example usage (for demonstration)
if __name__ == "__main__":
    #...
    m2 = Module("AABD", "Database architecture and administration", coef=2,
    credit=4, hours_tp=1.5,
        continous_percent=40, exam_percent=60)
    u11 = Unit("UEM11","UE Méthodologie",[m1, m2])
    m2.set_grade(tp=8, exam=6)
    print("Module average:", m2.name, m2.calculate_average())
    print("Unit average:", u11.calculate_average())
    print("Unit credits:", u11.calculate_credits())
```

8. Check the abstraction principle: can we define `Semester` as a subclass?

تحقق من مبدأ التجريد: هل يمكننا تعريف Semester كفئة فرعية؟

9. Check encapsulation in `Module` for `calculate_average()` and `calculate_credits()`, and modify them if necessary.

تحقق من مبدأ التغليف لدالتي حساب المعدل والرصيد، وعدّلهما إن اقتضى الأمر.

10. Override `calculate_average()` and `calculate_credits()` for `Unit`.

أعد تعريف (Override) دالتي حساب المعدل والرصيد لفئة Unit.

```python
def calculate_average(self):
    """Example of aggregation - combining module averages."""
    if not self._modules:
        return 0
    total = sum(m.calculate_average() * m.coef for m in self._modules)
    coef_sum = sum(m.coef for m in self._modules)
    return total / coef_sum

def calculate_credits(self):
    """Polymorphism: different modules could override this behavior."""
    if not self._modules:
        return 0
    avg = self.calculate_average()
    if avg >= 10 :
        total_credits = sum(m.credit for m in self._modules)
    else:
        total_credits = sum(m.calculate_credits() for m in self._modules)
    return total_credits
```

11. Define the `Semester` class and implement its methods. عرّف فئة السداسي`Semester` وأنجز دوالها.

12. Implement the whole logic of the GSI canvas to calculate students' averages and credits.
أنجز برنامجا كاملا لمنهاج GSI لحساب معدلات و أرصدة الطلبة.

Part 2: GIT Git الجزء الثاني: نظام التحكم في الإصدارات

Objectives : Understand the basics of Git and GitHub. GitHub. فهم أساسيات Git و

Data : M1 GSI canvas منهاج السنة الأولى ماستر تخصص هندسة أنظمة الإعلام الآلي

Tools : Git software / Github.com

Project Structure بنية المشروع

```
gsi_project/

   main.py
   academicelement.py
   module.py
   unit.py
   semester.py

   tests/
       test_module.py
       test_unit.py
       test_semester.py

   README.md
```

Steps الخطوات

Create a new Git repository إنشاء مستودع Git جديد

```
mkdir gsi_project
cd gsi_project
git init
```

Add a README file إضافة ملف README
Create a simple README.md:

```
# GSI Academic Project
This project models an academic structure (Modules, Units, Semesters) using Python OOP concepts.
```

Then add and commit it:

```
git add README.md
git commit -m "Add README file"
```

Create and work on branches إنشاء والعمل على الفروع
For each new file or feature:

```
git checkout -b feature/module
touch module.py
git add module.py
git commit -m "Add Module class skeleton"
git checkout main
git merge feature/module
```

Repeat for:

- `unit.py` → `feature/unit`
- `semester.py` → `feature/semester`
- `academicelement.py` → `feature/abstraction`
- `tests/` → `feature/tests`

Committing and pushing الحفظ والدفع إلى المستودع البعيد

```
git add .
git commit -m "Implement Unit class and inherit from AcademicElement"
git push origin feature/unit
```

Testing الاختبار

Use `pytest` for each file in the `tests` directory.

Git Collaboration التعاون عبر Git

- Students can clone each other's repositories. يمكن للطلبة استنساخ مستودعات بعضهم البعض.
- Use forks and pull requests to suggest improvements. استخدام fork و request pull لاقتراح تحسينات.
- Comment and review code in GitHub UI. إضافة تعليقات ومراجعة الشيفرة عبر واجهة GitHub.

Final Integration الدمج النهائي

- Merge all branches into `main`.
- Ensure all tests pass.
- Tag the final version.

```
git tag v1.0
git push origin v1.0
```

0.1 Home work الواجب المنزلي

- Read the Home work recommendations for Git. قراءة توصيات إعداد تقرير للواجب المنزلي الخاصة بمستودعات GIT.
- Use Git for your project. استعمل مستودع Git للمشروع.
- Reimplement the previous game using **OOP**. إعادة برمجة اللعبة السابقة باستخدام **OOP**.
- Create objects representing the game elements: إنشاء كائنات تمثل عناصر اللعبة:
  - **Player** البطل (Player)
  - **Monster** الوحش (Monster)
  - **Bullet** الطلقة (Bullet)
  - **Game Board** لوحة اللعب (Game Board)

- Organize object properties and behaviors using classes and objects.
- تنظيم خصائص الكائنات وسلوكياتها باستخدام الأصناف (Classes) والكائنات .(Objects)
- Preserve the prototype's functionality while developing it using an object-oriented design.
- الحفاظ على وظائف النموذج الأولي وتطويرها باستخدام التصميم الكائني.

Appendix: Master 1 GSI canvas ملحق: منهاج أولى ماستر هندسة أنظمة الإعلام الآلي

1- Semestre 1 :

Unité d'Enseignement	VHS	V.H hebdomadaire				Coeff	Crédits	Mode d'enseignement		Mode d'évaluation	
	15 sem	C	TD	TP	Autres			A Distance	En présentiel	Continu	Examen
UE fondamentales											
UEF11											
F111 : Réseaux des couches basses	67,5	1,5	1,5	1,5		3	6		x	40%	60%
F112 : Algorithmique Avancée et Complexité	45	1,5		1,5		2	4		x	40%	60%
UEF12											
F121 : Système d'exploitation	45	1,5	1,5			2	4		x	40%	60%
F122 : Architectures Modernes des Systèmes Informatiques	45	1,5		1,5		2	4		x	40%	60%
UE Méthodologie											
UEM11											
M111 : Architecture et administration des bases de données	45	1,5		1,5		2	4		x	40%	60%
M112 : Méthodes et Technologies d'Implémentation	45	1,5		1,5		3	5		x	40%	60%
UE Découverte											
UE D11											
D111: Systèmes de Communication Vocaux et Vidéos	45	1,5		1,5		2	2		x	40%	60%
UE Transversale											
UE T11											
T111: Cloud Computing	22,5	1,5				1	1	x			100%
Total Semestre 1	**360h**	**12**	**3**	**9**		**17**	**30**				

2- Semestre 2 :

Unité d'Enseignement	VHS	V.H hebdomadaire				Coeff	Crédits	Mode d'enseignement		Mode d'évaluation	
	15 sem	C	TD	TP	Autres			A Distance	En présentiel	Continu	Examen
UE fondamentales											
UEF21											
F211 : Les Middlewares pour les systèmes répartis	45	1,5	1,5	1,5		3	6		x	40%	60%
F212 : Modélisation et Architectures logicielles	45	1,5		1,5		2	4		x	40%	60%
UF22											
UF221: Réseaux de la couche IP	67,5	1,5		1,5		3	6		x	40%	60%
UF222: Introduction au Machine Learning	45	1,5		1,5		2	4		x	40%	60%
UE Méthodologique											
UEM21											
M211 : Infographie	45	1,5		1,5		2	3		x	40%	60%
M212 : Gestion de l'incertain	45	1,5	1,5			2	3		x	40%	60%
UE Découverte											
UE D21											
D211 : Internet of Things	45	1,5		1,5		2	3		x	40%	60%
UE Transversale											
UET21											
T211: Cybercriminalité	22,5	1,5				1	1	x			100%
Total Semestre 2	**360h**	**12**	**3**	**9**	**0**	**17**	**30**				